19/9/20 Pg-114

Page Date 2-24

Q. 1's complement operator
Q. How to remember precedence of operator
Q. 2 rows ' ; ' yourself.
Q. 2 matrices M_1 & M_2 are to be stored in arrays A & B respectively. Each array can be stored either in RMO or CMO in contiguous memory location. The time complexity of an algorithm to compute M_1 & M_2 will be
a) Best if A is in RMO, B in CMO.
b) Both are in RMO
c) Both in CMO
d) Independent of storage scheme

Q. Implementation choice for above Q

```
int x=5;
int *y=&x;
printf("%d", x);
printf("%u", &x);
printf("%u", y);
printf("%d", *y);
printf("%u", &y);
```

Q. a[3] is ____ & *(a+3) is ____

Q. Compare both the

```
main(){ int a[]={10,20,30,40,50,60};
int *p[]={a, a+1, a+2, a+3, a+4, a+5};
int **pp=p; pp++;
printf("%d, %d, %d", pp-p, *pp-a, **pp);
printf("%d, %d, %d", pp-p, *pp-a, **pp); ++**pp;
printf("%d, %d, %d", pp-p, *pp-a, **pp); ++**pp;
printf("%d, %d, %d", pp-p, *pp-a, **pp);
}
```

Scanned with CamScanner

Visualize char **argv;
int (*daytab)[13];
int *daytab[13];
void *comp()
char (*comp)()
char (*(*f())[])()
char (*(*p[3])())[5]
How to access nested structs
malloc() → call
free() → call
Dependson
Dependor
Subset of malloc/free is malloc/sbrk
How holes created
Malloc which memory management used
Use of IP
Functions
3 major things of SCL
int *p; p=0;
int *p=0;
for (;;) 1,2,3,4,5,6,7
struct node{
int val;
struct node *next; };
void rearrange(struct node *list){
struct node *p, *q;
int temp;
if(!list || !list->next) return;
p=list; q=list->next;
while(q){
temp = p->val; p->val=q->val;
q->val = temp; p=q->next;
q=p?p->next:0;

a) 1,2,3,4,5,6,7 (b) 2,1,4,3,5,7 (c) 1,3,2,5,4,7,6 (d) 2,3,4,5,6,7,1

Grade 2016

Q) Put node from last & put it to beginning

```
typedef struct node {
    int value;
    struct node * next;
} Node;
Node *move_to_front(Node *head) {
    Node *p, *q;
    if ((head == NULL) || (head->next == NULL))
        return head;
    q = NULL; p = head;
    while (p->next != NULL) {
        q = p;
        p = p->next;
    }
    return head;
}
```

Choose correct alternative to replace the blank

a) q = NULL; p->next = head; head = p;
b) q->next = NULL; head = p; p->next = head;
c) head = p; p->next = q; q->next = NULL;
d) q->next = NULL; p->next = head; head = p;

Q) Using recursion traverse SLL in
(i) Original order
(ii) Reverse order

Q) Reverse a SLL code

Grade 2017

Q) In worst case the # of comparisons needed to search a #SLL of length (n) for a given element is
a) $\log_2 n$ b) n/2 c) log2 d) n

Scanned with CamScanner

Let 'P' be a single LL, let Q be a pointer to an intermediate node 'x' in the list. What is the worst-case time complexity of the best-known algorithm to delete the node 'x' from the list?

draw

Adv - 2

Disadv.

How to traverse (loop of infinite loop)

What sentinel has

Explain concept of

```
struct item {
    int data;
    struct item *next;
};
int f(struct item *p) {
    return ((p==NULL || p->next==NULL ||
    ((p->data <= p->next->data) &&
    f(p->next)));
}
```

Doubly LL code

Insert node a) at beginning

b) at end

c) Intermediate

Stack - way(2)

Where used

How all DS implemented (imp)

Implement stack using arrays - push

- pop

order of complexity in array + LL implement

Scanned with CamScanner

- LL code
- Dig
- Where pushed element comes
- Code for push()
- Code for pop()
- Queue (2 ways)
- Pointers
- Stack - both direction
- Queue - " "
- Problem
- Representation - circular array
- Eg
- Visualize
- Which type of incrementing used
- General
- ++
- How deleted
- Code of enqueue
- Code of dequeue()

Suppose a circular queue of capacity (n-1) elements is implemented with an array of 'n' elements. Assume that the insertion & deletion operations are carried out using 'REAR' & 'FRONT' as array index variables respectively. Initially REAR = FRONT = 0. The conditions to detect queue full & queue empty are:

Scanned with CamScanner

Gate 2004

Q: A circular LL is used to represent a queue. A single variable 'p' is used to access the queue. To which node should 'p' point to such that both the operations enqueue & dequeue can be performed in constant time?

1994 Q: Which permutations can be obt in o/p (in same order) using a stack assuming that I/P is sequence 12345 in that order?

a) 3,4,5,1,2 (b) 3,4,5,2,1 (c) 1,5,2,3,4 (d) 5,4,3,1,2

Q: Priority queue → def"

Q: How we can mimic stack operation using priority queue?

1997 Q: A priority queue 'Q' is used to implement a stack 'S' that stores characters. PUSH(C) is implemented as INSERT(Q,C,K) where K is an appropriate key chosen by implementation. POP is implemented as DELETEMIN(Q). For a sequence of PUSH operations, the keys are in

a) Non-decreasing order
b) Non-increasing "
c) Strictly "
d) Strictly decreasing "

2005 Q: Let 'S' be a stack of size $n \geq 1$ starting with the empty stack. Suppose we push the first 'k' natural numbers in sequence & then perform 'k' pop operations. Assume that Push & Pop take 'x' seconds each, and 'y' seconds elapse

Gate 2005

btw the end of one such stack operation.
for $m \geq 1$, define the stack life of 'm' as the time elapsed from end of push(m) to that of pop operation that removes 'm' from S. The average stack life of an element of this stack is
Q. Visualize & draw dig, how to implement queue with 2 stacks.
Q. pop from 1st stack repeatedly until
Q. what if new element come to stack
Q. Rule to shift from S_1 to S_2
Q. Operations performed ..... for 1 element
Q. 2"
(Q)
Write Q is implemented using 2 stacks S_1 & S_2 as shown:

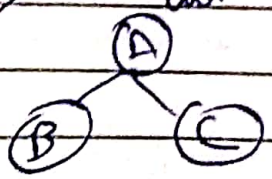
```
void insert (Q, x){
  push (S1, x);
}
void delete (Q){
  if (stack-empty(S2)) then
  if (stack-empty(S1)) then{
    Printf ("Q is empty");
  return;
}
else while (!(stack-empty(S1))){
    x= pop(S1);
    push(S2, x);
}
  Pop (S2);
}
```

Scanned with CamScanner

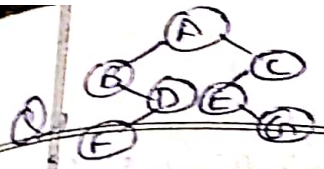
Page
Date

Gate 2000

- Suppose a stack implementation supports, in addition to PUSH & POP, an operation reverse that reverses the order of elements on the stack.
- Implement a queue using the above stack implementation, show how to implement "ENQUEUE" using a single operation & "DEQUEUE" using 3 operation
- Reverse way of above
- What is infix expression
- " " postfix
- Convert to postfix a) a+b
  b) a+b*c c) a+b-c
- Which DS used for infix → postfix
- What is pushed in stack
- Convert using stack a+(b-d)
  a+b*c
  a*b+c
  a*b+(c-d)
- Precedence of + C
  C +
- Name full algorithm for conversion
- Steps of algorithm for postfix evaluation
- Name of stack used
- Apply algo on 3+2*1
- 3 types of tree traversing - Name
  - Order all 3
- Traverse - short cut - 1st time, 2nd time, 3rd
- Traverse



Scanned with CamScanner

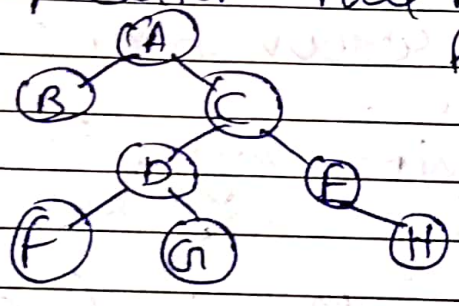


Page Date 13.2

- LL list node code
- Inorder code
- Explain using tree
- How to find space complexity
- Double order traversal - code
- Triple order traversal - code
- What is indirect recursion on trees
- Code of A & B for
- Apply indirect recursion using
  a) A first B) B first
- # of binary trees possible with n (unlabeled nodes)
- Derive proper →
- " same instead for labelled nodes
- Tell the binary trees with 3 nodes A, B, C which have preorder ABC
- ↑ + Postorder CBA
- ↑ + # Inorder BCA
- Conclusion always remember
- If Pre, Post, in given the # of binary tree
- " # of trees
- " Post, " "
- " In " "
- If In, Pre # of trees
- " In, Post " "
- Post, Pre " "

Scanned with CamScanner

Pre → ABC In → BAC Draw
What if # of nodes are large (Method 1)
In: 1 2 3 4 5 6 7 8
Pre: 5 3 1 2 4 6 8 7 , Find Post & draw tree
In: 1 2 3 4 5 6 7 8
Pst: 2 1 4 3 7 8 6 5
Logic to count # of leaves
Code
Count # of non leaves - code
Time complexity
Space "
What is full node
Relation full Node & Non leaves



Find # of
a) leaves
b) Non leaves
c) full nodes

Logic for FN
Code
Time & space complexity
What is height of node
Logic
Code
Binary tree diagram
Solution
O( )
Insert 50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24 into BST
Rules of BST

Scanned with CamScanner

Page Date 13/7

Q 2 properties of BST

Gate 2005 Q Postorder of BST: 10, 9, 23, 22, 27, 25, 15, 50, 95, 60, 40, 29 Find Inorder & Preorder & draw tree

Gate 2004 Q How many BSTs are possible with 4 distinct keys

Gate 2008 Q 1 C1=100 in BST. Search for 55. Which of the following sequences can't be sequence of nodes

a) 10, 75, 64, 43, 60, 57, 55
b) 90, 12, 68, 34, 62, 45, 55
c) 9, 85, 47, 68, 43, 57, 55
d) 79, 14, 72, 56, 16, 53, 55

Q Time to search in BST - worst case

Q So what we need

Gate 2005 Q A BST stores value in range 37 to 573 Consider the following sequence of keys

I) 81, 537, 102, 439, 285, 376, 305
II) 52, 97, 121, 195, 242, 381, 472
III) 142, 248, 520, 386, 345, 270, 307
IV) 550, 149, 507, 395, 463, 402, 270

A Now if 273 is inserted which is unsuccessful

B a) I, II, IV are inorder sequences of 3 diff BSTs
b) I is preorder sequence of some BST with 439 as root
c) II is an inorder of some BST with 121 as root and 52 as leaf
d) IV is a postorder of some BST with 149 as root

Gate 2008 Q I: MBCAFHPYK Find which is/are, Post out of I, II, III
II: KAMCBYPFH
III: MABCKYFPH

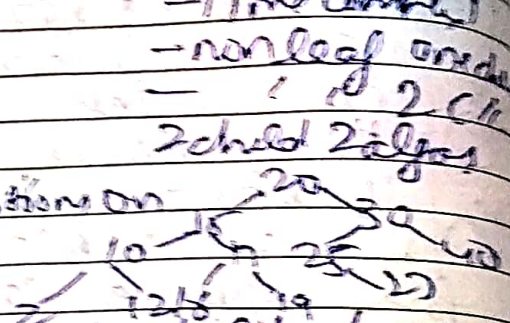
Gate 2005 Q We are given set of n distinct elements & an unlabelled tree with n node. In how many ways we can populate the tree if BST form

a) 0 b) 1 c) n! d) $2^n(n!/(n+1))$

Page
Date

Deleting a node from BST - Cases
- Tree Complete
- non leaf node
- 1 / 2 Children
- 2 child 2 edges

Implement above Question on



Q: Height change in all delete operations
Q: Inorder successor meaning
Q: " predecessor "
Q: 2 things we have to find before delete
Q: Node 1st
2nd

Gate 2018 A complete n-ary tree is one in which every node has '0' or 'n' sons. If 'x' is the # of internal nodes of a complete n-ary tree the # of leaves in it is given by
a) x(n-1)+1 b) xn-1 c) xn+1 d) x(n+1)

Q: Side dig - heap & priority

Gate 2016 # of leaf nodes in a rooted tree of 'n' nodes, with each node having 0 or 3 children is
a) n/2 b) (n-1)/3 c) (n-1)/2 d) (2n+1)/3

Q: What is complete binary tree

Q: What is full binary tree

Q: Recursive prog to check if complete binary tree

Q: Notes

Q: New topic introduction

Scanned with CamScanner

Page Date 36

3 types of balanced BST & short notes
Balanced factor formula
Rotation Name & type of imbalance
Photo LL imbalance
" Rotation
" Photo RR
" Rotation
" Photo RL
" Rotation
" Photo LR
" Rotation
Insert in AVL 50, 20, 60, 10, 8, 15, 32, 46, 11, 48
BST - search - Time complexity
BST - height
" - Iteration
AVL - search
" - height
" - Iteration
why O(log n) + O(log n) + C
What is C

Insert into AVL 21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7
Max nodes in AVL of height n derive
Result
Min " "
Result

A binary tree 'T' has 'n' leaf nodes. The # of nodes of degree '2' in T is
a) $\log_2 n$ b) $n-1$ c) n d) 2^n

In a binary tree # of internal nodes of degree '1' is 5, & # of internal nodes of degree 2 is 10. The # of leaf nodes are
a) 10 b) 11 c) 12 d) 15

Scanned with CamScanner

Q. General derive for previous ques
Diff by degree 1

gate 2008 Q. True statement about search times

	AVL	BST
a)	$\Theta(\log n)$	$O(n)$
b)	$\Theta(\log n)$	$\Theta(n \log n)$
c)	$\Theta(n)$	$\Theta(\log n)$
d)	$\Theta(n \log n)$	$\Omega(n)$

Q. Θ meaning
Q. Ω ,,

Gate 1998 Q. Which is false
a) A tree with 'n' nodes has (n-1) edges.
b) A labeled rooted binary tree can be uniquely constructed given its Post & preorder traversal
c) A complete binary tree with 'n' internal nodes has n+1 leaves
d) Max # of nodes in a binary tree of height h is $2^{h+1}-1$

Gate 2005 Q. In a binary tree, for every node the diff b/w the # of nodes in the left & right subtrees is atmost 2. If the height of the tree is $h>0$, then the min # of nodes in the tree is
a) 2^{h-1} b) $2^{h-1}+1$ c) 2^h-1 d) 2^h

1997 Q. N(H) for AVL tree
Q. A size balanced binary tree is a binary tree in which for every node, the difference b/w the # of nodes in left & right subtree is atmost 1. The distance of a node from the root is length of path from the root to node. The height of a binary tree is the maximum distance of a leaf node from root

A) Prove by simple + induction both on h, that a size-balanced binary tree of height 'h' contains at least 2^h nodes

B) Max height of such a tree with 'n' nodes

Gate 2013 C) Suppose we have a balanced binary search tree 'T' holding 'n' numbers. We are given 2 numbers 'L' & 'H' & wish to sum up all the numbers in 'T' that lie bw 'L' & 'H'. Suppose there are 'm' such numbers in T. If the tightest upper bound on time to compute the sum is $O(n^a \log^b n + m^c \log^d n)$, the value of $a+10b+100c+1000d$ is

D) other way

E) what is an expression tree

F) Draw exp tree for

a) a+b

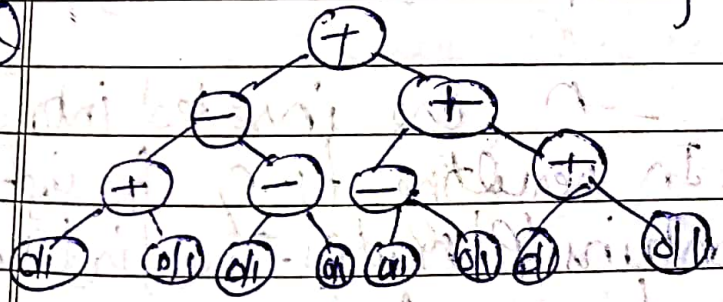
b) a+b* c and do pre, in, post traversal

c) 3 types of expressions

d) Exp tree for $-a$
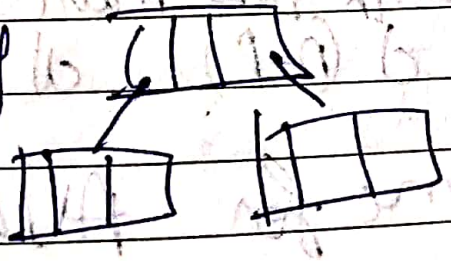
$a^{\log a}$

$\log a^{a!}$



Find max value

Disadv of



Representation of binary tree

Scanned with CamScanner

Page
Date

- 2nd representation – name & extend
- Draw LCRS for 
- 3rd representation
- Root index
- left "
- Right "
- Why 10 indexes
- How to get Parent from children.
- Why so formula
- Where array representation used
- ↓ → → not ↓
- eg diy
- eg diy
- So conclusion when advisable
- Another way
- 2 methods of nested form
- Retrieval ... in ↓ ↓ (Root L R)
- 4 types to for & where used
- Gate 2006 (x, y, z) y & z can be null, which is Root L R valid binary tree

a) (1 2 (4 5 6 7)) b) (1 ((2 3 4) 5 6) 7)
c) (1 (2 3 4) (5 6 7)) d) (1 (2 3 NULL) (4 5))

Gate 2005 The numbers 1, 2, 3 ---- n are inserted into a BST in some order. In resulting tree, the right subtree of root contains 'p' nodes. The first number to be inserted in tree must be

a) p b) p+1 c) (n-p) d) (n-p+1)

- Calculate same for AVL BST

Scanned with CamScanner

Page
Date

Q. Ceil & floor notation

Q) An array 'x' of 'n' distinct integers is interpreted as a complete binary tree. The index of the first element of array is 0. The index of parent of element x[i], i≠0 is

a) $\lfloor \frac{i}{2} \rfloor$ b) $\lceil \frac{i-2}{2} \rceil$ c) $\lfloor \frac{i}{2} \rfloor$ d) $\lceil \frac{i}{2} \rceil - 1$

b) If the root node is at level 0, the level of element x[i] i≠0 is

a) $\lfloor \log_2 i \rfloor$ b) $\lceil \log_2 (i+1) \rceil$ c) $\lfloor \log_2 (i+1) \rfloor$ d) $\lceil \log_2 i \rceil$

Q) leftmost right sibling is used. Each node of tree is of type treeNode.

```
typedef struct treeNode *treePtr;
struct treeNode {
    treePtr leftMostChild, rightSibling;
};
int DoSomething (treePtr tree) {
int value=0;
if (tree != NULL) {
if (tree -> leftMostChild == NULL)
    value = 1;
else {
value = DoSomthg (tree -> leftmostChild);
value = value + DoSomthing (tree -> rightSibling);}}
return value;
}
```

a) function will call # of internal nodes

b) " " " return height of tree

c) " " count # of right sibling nodes

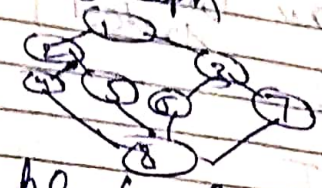
d) # of leaf nodes count.

Page
Date

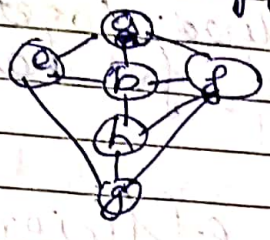
Q What is graph theory
Q Eg's - 2
Q Search
Q Traversal
Q BFS at level 1 for
Q what is DFS
Q Name 2 ways of representing graph
Q Adjacency matrix draw for
Q Order of space complexity
Q what if weighted graph
Q Adjacency list for
Q Order
Q Which used in dense graph
Q ——— Plane ?
Q What is visited
Q " " " explored
Q table possible & not possible comb's
Q DS for visited
Q DS for explored - 2
Q When queue used ——
Q " Stack used ——
Q Order of complexities
Q BFS Time & Space adjacency matrix
Q ——— " " list
Q Can a graph be disconnected
Q what if disconnected
Q How to know if connected or not
Q Code of BFS / DFS
Q Run on
Q Represent in adjacency list
Q Note
Q In mature

Scanned with CamScanner

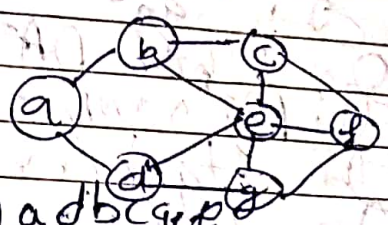
Q) Complexities comparison both DFS & BFS
Q) What if directed graph
Q) DFS code
Q) Traverse by DFS
Q) Visualize stacks
Q) Time & space complexity but Matrix
Q) Possible sequence if DFS applied



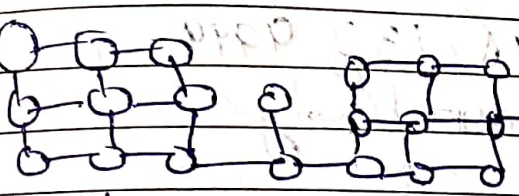
(I) abegbf
II abfegb
III abfhgfe
IV afghbe



Q) Not possible DFS of
1) abefdgc 2) abefcgd 3) abgebcf 4) adbcgef



Q) DFS, find worst case stack complexity



Consider a DFS of an undirected graph with 3 vertices P, Q & R. let discovery[u] represent the time instant when the vertex 'u' is first visited, & finish time f[u] represent time instance when vertex u is last visited. Given that

d(P) = 5 units f(P) = 12 units
d(Q) = 6 " f(Q) = 10 "
d(R) = 14 " f(R) = 18 "

What is true.
— is — representation in above Q
DFS major operation
Most used in static dataset

Scanned with CamScanner

Page
Date

Q searching time complexities
1) Array (unsorted)
2) " (sorted)
3) LL
4) BT
5) BST
6) AVL
7) Priority Queue

Q what introduced to get sol^n of & time complexity
Q what is used for hashing
Q expand DAT
Q eg
Q when DAT efficient - 2
Q Disadv - 2
Q Hashing - assumptions eg
Q Diff
Q - Insert 121, 146, 132, 9999
Q Inserting time complexity
Q Searching " "
Q problem
Q - eg 121, 132
Q - 3 sol's to problem
Q Disadv of 1
Q Diff of 2
Q Better for
Q Diff of 3
Q Better for
Q 3 types of 3
Q 2 diff bw chaining & ~~hashing~~ open addressing
Q Chang Total Time = ___ + ___
Q Explain how " constant

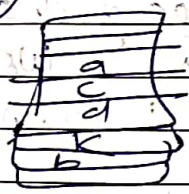
Scanned with CamScanner

Worst case
Average in search formula
What is load factor
n =
m =
average deletion time
Open addressing comparison
Disadv chaining
Adv of chained hash table over open addressing is worst case complexity of search is less.
b) Space used is less deletion is easier in chaining

h(k) = k mod 9 has 9 slots, chaining is used
keys 5, 28, 19, 15, 20, 33, 12, 17 & 10. Then max, min average chain lengths in hash table is

Consider a hash table with 'n' buckets, where chaining is used to resolve collisions. The hash function is the probability that a key value is hashed to a particular bucket is (1/n). The hash table is initially empty & 'k' distinct values are inserted in the table
a) What is the probability that bucket number '1' is empty after k insertions
b) What is probability that no collisions has occured in any of the operation?
c) What is the probability that first collision occurs at kth insertion
Open addressing - other name
Q range
Compare with chain
where element saved
Compare 1 & m.

Scanned with CamScanner

What is Probing?
What if collision first time meeting
h: U →
h: U × ___ called 1st & 2nd probe
What is probing sequence
worst time to search
If table full then write hash function & probe sequence
Deletion disadv
Eg of " using


if (deleted
soln to above problem
Worst case & # of jumps to clarify
3 techniques of open addressing
linear probing h(k) formula
Write full algo using start, probe, size
of probe sequence possible
Advantages
Secondary clustering
Eg
Primary "
Ideal prob
Explain primary clustering two things
Primary clustering disadv

Gate 2008 Keys: 12, 18, 13, 2, 3, 23, 5 & 15
HTS = 10 ; h(k) = K mod 10 & linear probing
Gate 2008 HTS = 11 (0 to 10) h(K) = K mod 11
Eg: 43, 36, 92, 87, 11, 4, 71, 13, 14 linear probe
what is index into which last record is inserted.

Scanned with CamScanner

Page Date 46

Gate 2007

Q Consider a hash function that distributes keys uniformly. HTS = 20. After hashing of how many keys, will the probability that any new key hashed collides with an existing one exceed 0.5

a) 5 b) 6 c) 7 d) 10

Gate 2010

Q. h[k] = k mod 10, linear probing

A) a) 46, 42, 34, 52, 23, 33
b) 34, 42, 23, 52, 33, 46
c) 46, 34, 42, 23, 52, 33
d) 42, 46, 33, 23, 34, 52

B) How many insertion sequences are possible

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

Q Quadratic probing hash function
Q Compare with linear probing
Q How c_1, c_2 chosen
Q Advantage
Q Disadvantage — 2
Q Possible prob sequences
Q Double hashing formula
Q Adv — 2
Q Linear prob seq
Q Quad " "
Q Double " "
Q → why - 10
Q h_1, h_2 ?
Q Double hashing
Q P.S
Q How to make sure that ← happen.

Scanned with CamScanner